Öğrenme fazları: 1 • Ön-öğretim | 2 • İçerik | 3 • Katılım | 4 • Değerlendirme | 5 • İzleme

1 • Ön-öğretim **Ön Bilgi:** Hf.4'te `Koltuk` seçimini kendi `useState`'inde tuttu; `App/KoltukPlani` bundan habersizdi. `tipler.ts/veri.ts` ve `EtkinlikKarti/EtkinlikListesi/Bolum` Hf.4'ten aynen taşındı.

» Ünite 1 — `useEffect` ve Yaşam Döngüsü

Yan Etki (*Side Effect*)

Bir bileşenin render işinin (JSX üretmenin) dışında, dış dünyaya dokunan iş — belge başlığı, zamanlayıcı. Render sırasında doğrudan yapılırsa React bileşeni istediği kadar render edebildiğinden öngörülemez tekrarlara yol açar.

`useEffect` Çağrısı

`useEffect(() => {...}, [bagimliliklar])` — ilk argüman (fonksiyon) render tamamlandıktan sonra çalışır; ikinci argüman (bağımlılık dizisi) efektin ne zaman tekrar çalışacağını belirler.

Bağımlılık Dizisi: Üç Hâl

Dizi hiç verilmezse efekt her render'dan sonra çalışır; `[]` verilirse yalnız ilk render'dan (*mount*) sonra bir kez çalışır; `[deger]` verilirse yalnız o değer bir öncekine göre değiştiyse tekrar çalışır.

Temizleme (*Cleanup*) Fonksiyonu

Efekt fonksiyonunun geri döndürdüğü bir fonksiyon; React bunu bileşen kaldırılmadan (*unmount*) hemen önce ve efekt bağımlılık değişikliği yüzünden tekrar çalışmadan hemen önce çağırır.

Ortak Hata / Tuzak

İkinci argümanı (bağımlılık dizisini) tamamen atlayıp efekt içinde `setState` çağırmak sonsuz döngüye yol açar — her render efekti tekrar çalıştırır, efekt yeni bir `setState` tetikler.

Kalan saniyeyi tutup zamanlayıcıyı temizleyen özel hook (`use_geri_sayim_hook` etiketi):

```ts
// baslangicSaniye'den geriye sayar: useState ile kalanSaniye tutulur,
// useEffect içindeki setInterval her saniye setKalanSaniye'yi çağırır.
// Süre bitince (bittiMi) zamanlayıcı YENİDEN kurulmaz; efekt her
// çalıştığında (ve bileşen ekrandan kalktığında) döndürdüğü temizleme
// fonksiyonuyla clearInterval çağrılır — bu olmasaydı bileşen unmount
// olsa bile zamanlayıcı arka planda çalışmaya devam ederdi.
export function useGeriSayim(baslangicSaniye: number) {
  const [kalanSaniye, setKalanSaniye] = useState(baslangicSaniye);
  const bittiMi = kalanSaniye <= 0;

  useEffect(() => {
    if (bittiMi) return;

    const id = setInterval(() => {
      setKalanSaniye((onceki) => (onceki <= 1 ? 0 : onceki - 1));
    }, 1000);

    return () => clearInterval(id);
  }, [bittiMi]);

  return { kalanSaniye, bittiMi };
}
```

» Ünite 2 — Formlar ve Kontrollü Girdi

Kontrollü vs Kontrolsüz Girdi

Kontrolsüz girdide `<input>` değerini kendi DOM'unda tutar (React'in haberi yok); kontrollü girdide `value` her zaman React state'inden okunur — tek doğruluk kaynağı state'tir, DOM değil.

`value`/`onChange` Çifti

`<input value={ad} onChange={(e) => setAd(e.target.value)} />` — `value` verip `onChange` vermemek React'in "salt okunur girdi" uyarısı vermesine, kullanıcının hiçbir şey yazamamasına yol açar.

Tiplenmiş Form State'i

Birden fazla alan tek bir `interface` ile gruplanır ve tek bir `useState` ile tutulur; bir alanı güncellerken diğerlerini korumak için yayma operatörü kullanılır: `setForm({...form, ad: e.target.value})`.

`select` ve `checkbox`

`<select value={...} onChange={...}>` sınırlı bir seçenek kümesinden seçim yapar; `checkbox` `value` yerine `checked`, `e.target.value` yerine `e.target.checked` kullanır.

Ortak Hata / Tuzak

Doğrulama olmadan `onSubmit`'in ilk satırında `e.preventDefault()`'ı unutmak tarayıcının varsayılan davranışını (sayfayı yeniden yüklemek) tetikler — tüm React state'i sıfırlanır.

» Ünite 3 — Durum Kaldırma ve `Context`

Durum Kaldırma (*Lifting State Up*)

Birden fazla bileşenin paylaşması gereken bir durumu ortak en yakın üst bileşene taşıma; üst bileşen durumu tutar, alt bileşenlere güncel değeri ve bir callback'i prop olarak geçirir.

Prop Drilling Sorunu

Durumu `App`'e kadar kaldırdığınızda aradaki bileşenler (`KoltukPlani`) o veriyle hiçbir iş yapmasa bile onu sadece alt'a aktarmak için props'una eklemeniz gerekir; ağaç derinleştikçe bu sayı katlanarak büyür.

`createContext`/`useContext`

Bir Context, ağacın herhangi bir üst noktasında `<XContext.Provider value={...}>` ile yayınlanan ve herhangi bir alt noktadan, aradaki bileşenlere uğramadan, `useContext(XContext)` ile okunan bir veri kanalı.

`SepetContext` Tasarımı

Seçili koltuklar, toplam tutar, `koltukSec`, `sepetiTemizle` tek bir Context'te toplanır; `SepetProvider` bunları `useState` ile tutup `children`'a sağlar.

Ortak Hata / Tuzak

Bir bileşeni `<SepetProvider>...</SepetProvider>` sarmayı unutup `useContext` çağırmak `null` döndürür — bir bileşen kendi sağladığı Context'i doğrudan okuyamaz, yalnızca alt'ları okuyabilir.

İpucu

`useReducer`: birden çok ilişkili eylemi tek bir `sepetReducer(durum,eylem)` fonksiyonunda toplayan, `useState`'e alternatif araç — bu haftanın sepeti sadelik için `useState` kullanır.

» Ünite 4 — Özel Hook'lar ve Kompozisyon

Özel Hook (*Custom Hook*)

Adı `use` ile başlayan, içinde başka hook çağıran sıradan bir TypeScript fonksiyonu; state tutar/efekt çalıştırır ama JSX döndürmez — yalnız mantık paylaşır.

`useSepet()`

`useContext(SepetContext)`'i sarar; `null` dönerse (Provider dışında kullanılmışsa) açık `throw new Error(...)` fırlatır — hata hemen, anlaşılır ortaya çıkar.

Bileşen Bölme Ölçütü

(a) dosya okumakta zorlandığınız kadar uzuyorsa, (b) aynı JSX/mantık tekrarlanıyorsa, (c) bileşende ilgisiz iki sorumluluk varsa — bölme zamanı gelmiştir.

Ortak Hata / Tuzak

`useSepet()` içindeki `throw`'u silmek, Provider dışı kullanımda belirsiz bir çalışma zamanı hatasına (`Cannot read properties of null`) yol açar — neden anlaşılmaz.

» Komutlar

`npm run dev`	HMR ile anlık geliştirme sunucusunu localhost:5173'te açar
`npx tsc -b`	JS üretmeden tüm projeyi tip hatalarına karşı denetler
`npm run build`	tsc denetiminden geçirip optimize edilmiş dist/ üretir

5 • İzleme **Nerede Kullanılır:** Render dışı iş → `useEffect`+temizleme · Yazılanı anında görmek → `value`+`onChange` · aynı durumu paylaşan bileşenler → durum kaldırma; derin katman → `Context` · Tekrarlanan state+efekt → özel hook · Görünüm sarma → `children` · Çoklu ilişkili eylem → `useReducer`